

Atividade Prática - Programação Lógica

Bacharelado em Ciência da Computação

Prof. Luiz Eduardo da Silva

Nome: Isabella Cristina da Silveira RA: 2019.2.08.004

Índice

1 Atividade 1 – Ordenação de listas de listas	1
1.1 Parte A – Ordenação pelo tamanho das sublistas	1
1.1.1 Exemplo	2
1.1.2 Tarefas	2
1.2 Parte B – Ordenação pela frequência dos comprimentos	2
1.2.1 Exemplo	2
1.2.2 Explicação	3
1.2.3 Tarefas	3

1 Atividade 1 – Ordenação de listas de listas

Nesta atividade, considere que uma lista contém outras listas como elementos. O objetivo é implementar predicados em Prolog para ordenar essas sublistas segundo diferentes critérios.

1.1 Parte A – Ordenação pelo tamanho das sublistas

Implemente o predicado:

Isort(InList, OutList)

que receba uma lista de listas (InList) e produza uma nova lista (OutList) ordenada de acordo com o comprimento das sublistas.

As sublistas mais curtas devem aparecer antes das mais longas.

1.1.1 Exemplo

?- Isort([[a,b,c],[d,e],[f,g,h],[d,e],[i,j,k,l],[m,n],[o]], L).

Resultado esperado: L = [[o], [d,e], [d,e], [m,n], [a,b,c], [f,g,h], [i,j,k,l]]

1.1.2 Tarefas

1. Crie um predicado que calcule o tamanho de uma lista.
2. Associe cada sublista ao seu comprimento.
3. Ordene as sublistas com base no comprimento.
4. Teste a solução com diferentes conjuntos de dados.

% Calcula o tamanho de uma lista

tamanho([],0).

tamanho([_|T],N) :-

 tamanho(T,N1),

 N is N1 + 1.

% Associa uma lista ao seu tamanho

associa_tamanho(L, Tam-L) :-

 tamanho(L,Tam).

% Converte todas as sublistas para Tam-Lista

cria_pares([],[]).

cria_pares([H|T],[Par|R]) :-

 associa_tamanho(H,Par),

 cria_pares(T,R).

% Remove os tamanhos após a ordenação

remove_tamanhos([],[]).

remove_tamanhos([_Lista|T],[Lista|R]) :-

 remove_tamanhos(T,R).

% Predicado principal

Isort(InList,OutList) :-

 cria_pares(InList,Pares),

 keysort(Pares,Ordenados),

 remove_tamanhos(Ordenados,OutList).

EXPLICAÇÃO

Dada a lista: `[[a,b,c],[d,e],[f,g,h],[d,e],[i,j,k,l],[m,n],[o]]`

Os tamanhos são:

`[a,b,c]` -> 3

`[d,e]` -> 2

`[f,g,h]` -> 3

`[d,e]` -> 2

`[i,j,k,l]` -> 4

`[m,n]` -> 2

`[o]` -> 1

Ordenando pelo tamanho: `[[o],[d,e],[d,e],[m,n],[a,b,c],[f,g,h],[i,j,k,l]]`

Predicado tamanho/2 -

Recebe uma lista e devolve seu tamanho.

Exemplo:

?- tamanho([a,b,c,d],N).

N = 4.

Funcionamento:

tamanho([],0).

Lista vazia tem tamanho 0.

Para qualquer outra lista:

tamanho(_|T,N)

Ignora o primeiro elemento (`_`) e conta o resto.

Exemplo: `[a,b,c]` vira `[b,c]`, depois `[c]`, depois `[]`,

Somando 1 a cada passo.

—

Predicado cria_pares/2

Transforma: $[[a,b,c],[d,e],[o]]$ em $[3-[a,b,c],2-[d,e],1-[o]]$

Assim podemos ordenar usando o número como chave.

Predicado keysort/2

Já existe no Prolog.

Ordena pela chave: $[3-[a,b,c],2-[d,e],1-[o]]$ vira $[1-[o],2-[d,e],3-[a,b,c]]$

remove_tamANHos/2

Remove as chaves: $1-[o]$ fica $[o]$

1.2 Parte B – Ordenação pela frequência dos comprimentos

Implemente o predicado: **lfsort(InList, OutList)**

que ordene as sublistas de acordo com a frequência com que seus comprimentos aparecem na lista original.

A ideia é que comprimentos mais raros apareçam primeiro.

1.2.1 Exemplo

?- lfsort($[[a,b,c],[d,e],[f,g,h],[d,e],[i,j,k,l],[m,n],[o]]$, L).

Resultado esperado: $L = [[i,j,k,l], [o], [a,b,c], [f,g,h], [d,e], [d,e], [m,n]]$

1.2.2 Explicação

- Comprimento 4 → aparece 1 vez.
- Comprimento 1 → aparece 1 vez.
- Comprimento 3 → aparece 2 vezes.
- Comprimento 2 → aparece 3 vezes.

Portanto, as listas de comprimentos menos frequentes aparecem primeiro.

1.2.3 Tarefas

1. Determine o comprimento de cada sublista.
2. Calcule a frequência de cada comprimento.
3. Associe cada sublista à frequência do seu comprimento.
4. Ordene as sublistas pela frequência (ordem crescente).
5. Compare os resultados obtidos por Isort/2 e lfsort/2.

EXPLICAÇÃO

O que deve fazer?

Agora não importa apenas o tamanho. Importa quantas vezes aquele tamanho aparece.

Lista: [[a,b,c],[d,e],[f,g,h],[d,e],[i,j,k,l],[m,n],[o]]

Tamanhos: 3, 2, 3, 2, 4, 2, 1

Frequências:

1 -> aparece 1 vez

2 -> aparece 3 vezes

3 -> aparece 2 vezes

4 -> aparece 1 vez

Então:

comprimento 4 -> frequência 1

comprimento 1 -> frequência 1

comprimento 3 -> frequência 2

comprimento 2 -> frequência 3

Logo: [[i,j,k,l],[o],[a,b,c],[f,g,h],[d,e],[d,e],[m,n]]

Passo 1

Extrair os tamanhos.

Entrada: [[a,b,c],[d,e],[f,g,h],[d,e],[i,j,k,l],[m,n],[o]]

Saída: [3,2,3,2,4,2,1]

Passo 2

Contar frequências.

3 -> 2 vezes

2 -> 3 vezes

4 -> 1 vez

1 -> 1 vez

Passo 3

Criar pares.

[2-[a,b,c], 3-[d,e], 2-[f,g,h], 3-[d,e], 1-[i,j,k,l], 3-[m,n], 1-[o]]

Passo 4

Ordenar.

[1-[i,j,k,l], 1-[o], 2-[a,b,c], 2-[f,g,h], 3-[d,e], 3-[d,e], 3-[m,n]]

Passo 5

Remover as frequências.

Resultado: [[i,j,k,l], [o], [a,b,c], [f,g,h], [d,e], [d,e], [m,n]]

RESUMO

O que é o objetivo do trabalho?

O trabalho pede para ordenar listas de listas.

Exemplo: [[a,b,c],[d,e],[o]]

Essa é uma lista que contém outras listas.

Cada elemento da lista principal é uma sublista.

O objetivo é criar duas ordenações diferentes:

Isort (Ordenar pelo tamanho da sublista.)

Ifsort (Ordenar pela frequência dos tamanhos.)

O que é um predicado em Prolog?

Em Prolog, um predicado é parecido com uma função.

Exemplo: tamanho(Lista,Tam)

Recebe uma lista e retorna seu tamanho.

Consulta: ?- tamanho([a,b,c],N).

Resultado: N = 3.

Como funciona tamanho/2?

Código:

tamanho([],0).

tamanho([_|T],N) :-

tamanho(T,N1),

N is N1 + 1.

Primeira regra

tamanho([],0).

Significa: Se a lista estiver vazia, seu tamanho é 0.

Esse é o caso base.

Sem ele a recursão nunca terminaria.

Segunda regra

`tamanho([_|T],N)`

Que significa:

Uma lista pode ser dividida em: [Cabeça | Cauda]

Exemplo: `[a,b,c]` fica `a` -> cabeça - `[b,c]` -> cauda

No código `(_|)` quer dizer: "não me importo com esse valor".

Então: `[_|T]` significa: "Pega qualquer elemento e guarda o resto em T."

Exemplo passo a passo

Professor pode perguntar:

Como o Prolog calcula `tamanho([a,b,c],N)`?

Resposta:

Primeira chamada:

`tamanho([a,b,c],N)`

vira:

`tamanho([b,c],N1)`

Depois:

`tamanho([c],N2)`

Depois:

`tamanho([],N3)`

Chega no caso base:

$N3 = 0$

Voltando:

$N2 = 1$

Voltando:

$N1 = 2$

Voltando:

$N = 3$

Resultado:

$N = 3$

O que é recursão?

Recursão acontece quando um predicado chama ele mesmo.

Exemplo:

`tamanho([_|T],N) :-`

`tamanho(T,N1),`

`N is N1 + 1.`

O predicado tamanho chama novamente o próprio tamanho.

Isso é recursão.

O que faz associa_tamanho?

Código:

`associa_tamanho(L,Tam-L)`

Recebe: [a,b,c]

Calcula: 3

E cria: 3-[a,b,c]

Por que criar 3-[a,b,c]?

Porque o Prolog sabe ordenar números.

Então transformamos cada lista em um par: Tamanho-Lista

Exemplo:

[3-[a,b,c], 2-[d,e], 1-[o]]

Agora ficou fácil ordenar.

O que faz cria_pares?

Transforma: [[a,b,c], [d,e], [o]] em [3-[a,b,c], 2-[d,e], 1-[o]]

O que é keysort?

Essa é uma pergunta que eu apostaria que pode cair.

O keysort é um predicado pronto do Prolog.

Ele ordena pares do tipo: Chave-Valor

Exemplo: [3-a, 1-b, 2-c]

Após: keysort(...)

vira: [1-b, 2-c, 3-a]

Ele usa apenas a parte da esquerda.

Como funciona o Isort?

Passo 1:

Recebe: [[a,b,c], [d,e], [o]]

Passo 2:

Calcula os tamanhos.

3

2

1

Passo 3:

Cria pares.

[3-[a,b,c], 2-[d,e], 1-[o]]

Passo 4:

Ordena.

[1-[o], 2-[d,e], 3-[a,b,c]]

Passo 5:

Remove os números.

[[o], [d,e], [a,b,c]]

Agora o lfsort

Esse é o mais importante.

Muita gente entende o lsort mas se confunde no lfsort.

Qual a diferença?

No lsort: ordena pelo tamanho.

No lfsort: ordena pela frequência do tamanho.

Exemplo

Lista: [[a,b,c], [d,e], [f,g,h], [d,e], [i,j,k,l], [m,n], [o]]

Tamanhos

3 2 3 2 4 2 1

Frequência

Professor pode perguntar:

O que é frequência?

Resposta:

É o número de vezes que um tamanho aparece.

Aqui:

1 aparece 1 vez

2 aparece 3 vezes

3 aparece 2 vezes

4 aparece 1 vez

Então cada sublista recebe uma frequência

[a,b,c] -> tamanho 3 -> frequência 2

[d,e] -> tamanho 2 -> frequência 3

[f,g,h] -> tamanho 3 -> frequência 2

[i,j,k,l] -> tamanho 4 -> frequência 1

[o] -> tamanho 1 -> frequência 1

Criamos os pares

[2-[a,b,c], 3-[d,e], 2-[f,g,h], 3-[d,e], 1-[i,j,k,l], 3-[m,n], 1-[o]]

Ordenando

[1-[i,j,k,l], 1-[o], 2-[a,b,c], 2-[f,g,h], 3-[d,e], 3-[d,e], 3-[m,n]]

Resultado final

[[i,j,k,l], [o], [a,b,c], [f,g,h], [d,e], [d,e], [m,n]]

Perguntas que eu faria se fosse o professor

O que é caso base?

É a condição que encerra a recursão.

Exemplo:

tamanho([],0).

O que é recursão?

É quando um predicado chama ele mesmo.

O que significa [_|T]?

Separar uma lista em: cabeça e cauda

O que faz o keysort?

Ordena pares do tipo: Chave-Valor, usando a chave.

Diferença entre lsort e lfsort?

lsort: Ordena pelo tamanho da sublista.

lfsort: Ordena pela frequência dos tamanhos.

Qual foi a parte mais importante do trabalho?

No `lsort`, associar cada lista ao seu tamanho.

No `lfsort`, calcular quantas vezes cada tamanho aparece e usar essa frequência como chave de ordenação.